RouteAI – Projektbeskrivning för Replit

> **Modernt AI-drivet planerings- och ruttoptimeringssystem**
> Version 1.0 | Fas 1 Implementation

---

1. Projektöversikt och Mål

1.1 Vad systemet ska göra

RouteAI är ett modernt, AI-drivet ruttoptimerings- och flottstyrningssystem som hjälper företag att planera, optimera och övervaka leveransrutter i realtid. Systemet kombinerar avancerad ruttoptimering med stöd för elfordon (EV), hållbarhetsrapportering och AI-assisterad beslutsfattning.

Kärnfunktionalitet:
- Automatisk ruttoptimering baserat på trafikmönster, tidsfönster och fordonskapacitet
- Stöd för blandade flottor (fossila fordon + elfordon)
- Realtidsspårning av förare och leveranser
- AI-assisterad planering med naturligt språk
- Hållbarhetsrapportering med CO2-beräkningar
- EV-specifik optimering med laddstationsplanering

1.2 Målgrupp

Segment	Beskrivning	Behov
Små leveransföretag	5-20 fordon, lokal distribution	Enkel ruttplanering, kostnadsbesparingar
Medelstora logistikföretag	20-100 fordon, regional täckning	Avancerad optimering, flottstyrning
E-handelsföretag	Last-mile delivery	Snabb optimering, kundnotifikationer
Serviceföretag	Fältservice, hemtjänst	Tidsfönsterhantering, kapacitetsplanering
Hållbarhetsfokuserade företag	ESG-rapportering	CO2-spårning, EV-optimering

1.3 Affärsvärde

- **25% lägre leveranskostnader** genom optimerade rutter
- **30% minskade CO2-utsläpp** med EV-optimering och smart planering
- **99% leveranser i tid** med AI-driven tidsberäkning
- **50% snabbare planering** jämfört med manuell ruttläggning
- **Ökad kundnöjdhet** genom exakta leveranstider och spårning

2. Kärnfunktioner för Fas 1

2.1 Prioritet 1 – Kritiska funktioner (MVP)

#	Funktion	Beskrivning
1.1	**Ruttoptimering**	Automatisk optimering av rutter baserat på stopp, tidsfönster och fordonskapacitet
1.2	**Interaktiv kartvy**	Leaflet-baserad karta med färgkodade rutter per förare
1.3	**Fordonshantering**	CRUD för fordon med stöd för både fossila och elfordon
1.4	**Stopphantering**	Lägg till, redigera och ta bort leveransstopp med adress och tidsfönster
1.5	**Drag-and-drop**	Manuell justering av rutter genom att dra stopp mellan förare
1.6	**Grundläggande dashboard**	Översikt med nyckeltal (antal stopp, total distans, beräknad tid)

2.2 Prioritet 2 – Viktiga funktioner

#	Funktion	Beskrivning
2.1	**Import/Export**	CSV-import av stopp, export av rutter till Excel/PDF
2.2	**Tidsfönsterhantering**	Stöd för leveranstidsfönster per stopp
2.3	**Kapacitetsplanering**	Viktbegränsningar och volymkapacitet per fordon
2.4	**Rutthistorik**	Spara och återanvänd tidigare rutter
2.5	**Grundläggande statistik**	Daglig/veckovis sammanställning av körda rutter

2.3 Prioritet 3 – Differentierade funktioner

#	Funktion	Beskrivning
3.1	**EV-batterioptimering**	Räckviddsberäkning och automatisk laddstationsplanering
3.2	**AI-chattassistent**	Naturligt språk för ruttfrågor (“Visa försenade leveranser”)
3.3	**CO2-dashboard**	Realtidsvisning av utsläpp och besparingar
3.4	**Scenarioplanering**	“Vad händer om”-simuleringar
3.5	**Prediktiv tidsberäkning**	ML-baserad ETA-beräkning

---

3. Teknisk Stack och Verktyg

3.1 Frontend

```
Framework:      Next.js 14+ (App Router)
Språk:          TypeScript
Styling:        Tailwind CSS
UI-komponenter: shadcn/ui
Animationer:    Framer Motion
Ikoner:         Lucide React
Formulär:       React Hook Form + Zod
State:          Zustand eller React Context
```

3.2 Backend

```
API:            Next.js API Routes (Route Handlers)
Validering:     Zod
Auth:           NextAuth.js (valfritt i Fas 1)
```

3.3 Databas

```
Databas:        PostgreSQL (via Neon, Supabase eller Railway)
ORM:            Prisma
Migrations:     Prisma Migrate
```

3.4 Kartor och Routing

```
Kartbibliotek:  Leaflet + React-Leaflet
Kartdata:       OpenStreetMap (gratis)
Geocoding:      Nominatim (OpenStreetMap) eller Mapbox
Routing:        OSRM (Open Source Routing Machine)
                - Self-hosted eller osrm.project-osrm.org
```

3.5 AI-integration

```
AI-provider:    Abacus.AI APIs
Användning:
  - Chattassistent för naturligt språk
  - Prediktiv tidsberäkning
  - Anomalidetektering
  - Scenarioanalys
```

3.6 Övriga verktyg

```
Grafer:         Recharts eller Tremor
Tabeller:       TanStack Table
Datum:          date-fns
Validering:     Zod
HTTP-klient:    Axios eller fetch
```

---

4. Designinsikter från Routific-analysen

4.1 Vad som fungerar bra (behåll)

✅ **Tydligt värdeerbjudande** – Konkret, mätbart (“25% lägre kostnader”)
✅ **Färgkodade rutter** – Varje förare har unik färg för enkel identifiering
✅ **Statistik-bar** – Snabb översikt (förare, stopp, km, tid) ovanför kartan
✅ **Alternerande layout** – Text/bild-sektioner skapar bra läsrytm
✅ **Dubbla CTA:er** – Fångar olika användartyper

4.2 Vad som ska moderniseras

🔄 **Interaktiv karta** – Animerade rutter istället för statiska bilder
🔄 **Dark mode** – Standard för dispatch-centraler och kontrollrum
🔄 **Mikro-animationer** – Hover-effekter, scroll-triggered animations
🔄 **Modern typografi** – Inter eller Satoshi istället för daterade fonts
🔄 **Glasmorfism** – Subtila frosted glass-effekter på kort

4.3 Färgschema (moderniserat)

Användning	Färg	HEX	Beskrivning
Primär bakgrund (dark)	Djup marinblå	`#0F172A`	Slate-900
Sekundär bakgrund	Mörkblå	`#1E293B`	Slate-800
Primär accent	Elektrisk cyan	`#06B6D4`	Cyan-500
Sekundär accent	Smaragdgrön	`#10B981`	Emerald-500 (EV/hållbarhet)
Varning	Amber	`#F59E0B`	Amber-500
Fel	Röd	`#EF4444`	Red-500
Text primär	Vit	`#F8FAFC`	Slate-50
Text sekundär	Grå	`#94A3B8`	Slate-400
Kort/Panels	Glasmorfism	`rgba(30,41,59,0.8)`	Med backdrop-blur

4.4 Typografi

```
/* Primär font */
font-family: 'Inter', system-ui, sans-serif;

/* Storlekar */
--text-xs: 0.75rem;    /* 12px */
--text-sm: 0.875rem;   /* 14px */
--text-base: 1rem;     /* 16px */
--text-lg: 1.125rem;   /* 18px */
--text-xl: 1.25rem;    /* 20px */
--text-2xl: 1.5rem;    /* 24px */
--text-3xl: 1.875rem;  /* 30px */
--text-4xl: 2.25rem;   /* 36px */

/* Vikter */
--font-normal: 400;
--font-medium: 500;
--font-semibold: 600;
--font-bold: 700;
```

4.5 Layout-principer

- **Border-radius:** 12px för kort, 8px för knappar, 6px för inputs
- **Spacing:** 8px grid-system (8, 16, 24, 32, 48, 64)
- **Skuggor:** Subtila, mjuka skuggor med blur
- **Glasmorfism:** `backdrop-filter: blur(12px)` på kort

---

5. Innovativa Funktioner (Differentiering)

5.1 EV-optimering

Funktion	Beskrivning
Räckviddsberäkning	Beräkna återstående räckvidd baserat på batterikapacitet, last och terräng
Laddstationsplanering	Automatisk inplanering av laddpauser vid behov
Energikostnadsoptimering	Planera laddning vid lägsta elpris (time-of-use)
Blandad flotta	Optimera rutter för både fossila och elfordon samtidigt
Batteristatusvisning	Realtidsvy av batteriläge för alla EV i flottan

5.2 AI-funktioner

Funktion	Beskrivning
Naturligt språk-gränssnitt	“Visa alla försenade leveranser idag”
Prediktiv ETA	ML-baserad leveranstidsberäkning
Anomalidetektering	Identifiera avvikelser i leveranstider
Scenarioplanering	“Vad händer om vi lägger till 2 elbilar?”
Automatisk kapacitetsplanering	AI föreslår antal fordon baserat på ordermönster
Smart ruttförslag	AI-genererade optimeringsförslag

5.3 Hållbarhetsrapportering

Funktion	Beskrivning
CO2-dashboard	Realtidsvisning av utsläpp per rutt/förare/dag
Jämförelse fossil vs. el	Visa besparingar vid EV-användning
ESG-rapporter	Exporterbara hållbarhetsrapporter
Träd-ekvivalent	Visualisera CO2-besparing som planterade träd
Trendanalys	Visa utsläppsutveckling över tid

---

6. Databas-schema

6.1 ER-diagram (Mermaid)

```
erDiagram
    ORGANIZATION ||--o{ USER : has
    ORGANIZATION ||--o{ VEHICLE : owns
    ORGANIZATION ||--o{ ROUTE : creates

    USER ||--o{ ROUTE : creates

    VEHICLE ||--o{ ROUTE : assigned_to
    VEHICLE }o--|| VEHICLE_TYPE : is_type

    ROUTE ||--o{ STOP : contains
    ROUTE ||--o{ ROUTE_HISTORY : has

    STOP }o--|| CUSTOMER : belongs_to

    ORGANIZATION {
        uuid id PK
        string name
        string email
        timestamp created_at
    }

    USER {
        uuid id PK
        uuid organization_id FK
        string email
        string name
        string role
        timestamp created_at
    }

    VEHICLE {
        uuid id PK
        uuid organization_id FK
        string name
        string license_plate
        enum vehicle_type
        float capacity_kg
        float capacity_volume
        float battery_capacity_kwh
        float current_battery_percent
        float range_km
        boolean is_active
        timestamp created_at
    }

    VEHICLE_TYPE {
        string type PK
        string description
    }

    ROUTE {
        uuid id PK
        uuid organization_id FK
        uuid vehicle_id FK
        uuid created_by FK
        string name
        date route_date
        enum status
        float total_distance_km
        int total_duration_min
        float estimated_co2_kg
```

```
        jsonb optimized_order
        timestamp created_at
    }

    STOP {
        uuid id PK
        uuid route_id FK
        uuid customer_id FK
        int sequence_order
        string address
        float latitude
        float longitude
        time time_window_start
        time time_window_end
        time estimated_arrival
        time actual_arrival
        enum status
        float weight_kg
        string notes
        timestamp created_at
    }

    CUSTOMER {
        uuid id PK
        uuid organization_id FK
        string name
        string email
        string phone
        string default_address
        float default_lat
        float default_lng
        timestamp created_at
    }

    ROUTE_HISTORY {
        uuid id PK
        uuid route_id FK
        jsonb route_snapshot
        string change_type
        timestamp created_at
    }
```

6.2 Tabellbeskrivningar

`organizations`

```
CREATE TABLE organizations (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    name VARCHAR(255) NOT NULL,
    email VARCHAR(255) UNIQUE NOT NULL,
    settings JSONB DEFAULT '{}',
    created_at TIMESTAMP DEFAULT NOW(),
    updated_at TIMESTAMP DEFAULT NOW()
);
```

`users`

```
CREATE TABLE users (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    organization_id UUID REFERENCES organizations(id) ON DELETE CASCADE,
    email VARCHAR(255) UNIQUE NOT NULL,
    name VARCHAR(255) NOT NULL,
    password_hash VARCHAR(255),
    role VARCHAR(50) DEFAULT 'dispatcher', -- admin, dispatcher, driver
    is_active BOOLEAN DEFAULT true,
    created_at TIMESTAMP DEFAULT NOW(),
    updated_at TIMESTAMP DEFAULT NOW()
);
```

`vehicles`

```
CREATE TABLE vehicles (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    organization_id UUID REFERENCES organizations(id) ON DELETE CASCADE,
    name VARCHAR(100) NOT NULL,
    license_plate VARCHAR(20),
    vehicle_type VARCHAR(20) NOT NULL, -- 'fossil', 'electric', 'hybrid'
    capacity_kg DECIMAL(10,2) DEFAULT 0,
    capacity_volume_m3 DECIMAL(10,2) DEFAULT 0,
    -- EV-specifika fält
    battery_capacity_kwh DECIMAL(10,2),
    current_battery_percent DECIMAL(5,2),
    range_km DECIMAL(10,2),
    charging_speed_kw DECIMAL(10,2),
    -- Allmänt
    color VARCHAR(7) DEFAULT '#3B82F6', -- HEX-färg för kartvisning
    is_active BOOLEAN DEFAULT true,
    created_at TIMESTAMP DEFAULT NOW(),
    updated_at TIMESTAMP DEFAULT NOW()
);
```

`customers`

```
CREATE TABLE customers (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    organization_id UUID REFERENCES organizations(id) ON DELETE CASCADE,
    name VARCHAR(255) NOT NULL,
    email VARCHAR(255),
    phone VARCHAR(50),
    default_address TEXT,
    default_latitude DECIMAL(10,8),
    default_longitude DECIMAL(11,8),
    notes TEXT,
    created_at TIMESTAMP DEFAULT NOW(),
    updated_at TIMESTAMP DEFAULT NOW()
);
```

routes

```
CREATE TABLE routes (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    organization_id UUID REFERENCES organizations(id) ON DELETE CASCADE,
    vehicle_id UUID REFERENCES vehicles(id) ON DELETE SET NULL,
    created_by UUID REFERENCES users(id) ON DELETE SET NULL,
    name VARCHAR(255),
    route_date DATE NOT NULL,
    status VARCHAR(20) DEFAULT 'draft', -- draft, optimized, in_progress, completed
    start_address TEXT,
    start_latitude DECIMAL(10,8),
    start_longitude DECIMAL(11,8),
    end_address TEXT,
    end_latitude DECIMAL(10,8),
    end_longitude DECIMAL(11,8),
    total_distance_km DECIMAL(10,2),
    total_duration_min INTEGER,
    estimated_co2_kg DECIMAL(10,2),
    route_geometry JSONB, -- GeoJSON för ruttlinjen
    optimized_stop_order UUID[], -- Array av stop-IDs i optimerad ordning
    created_at TIMESTAMP DEFAULT NOW(),
    updated_at TIMESTAMP DEFAULT NOW()
);
```

stops

```
CREATE TABLE stops (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    route_id UUID REFERENCES routes(id) ON DELETE CASCADE,
    customer_id UUID REFERENCES customers(id) ON DELETE SET NULL,
    sequence_order INTEGER NOT NULL,
    address TEXT NOT NULL,
    latitude DECIMAL(10,8) NOT NULL,
    longitude DECIMAL(11,8) NOT NULL,
    time_window_start TIME,
    time_window_end TIME,
    service_time_min INTEGER DEFAULT 5,
    estimated_arrival TIME,
    actual_arrival TIMESTAMP,
    status VARCHAR(20) DEFAULT 'pending', -- pending, arrived, completed, failed
    weight_kg DECIMAL(10,2) DEFAULT 0,
    volume_m3 DECIMAL(10,2) DEFAULT 0,
    notes TEXT,
    proof_of_delivery JSONB, -- { signature_url, photo_url, recipient_name }
    created_at TIMESTAMP DEFAULT NOW(),
    updated_at TIMESTAMP DEFAULT NOW()
);
```

`route_history`

```
CREATE TABLE route_history (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    route_id UUID REFERENCES routes(id) ON DELETE CASCADE,
    route_snapshot JSONB NOT NULL,
    change_type VARCHAR(50), -- created, optimized, manually_edited, completed
    changed_by UUID REFERENCES users(id),
    created_at TIMESTAMP DEFAULT NOW()
);
```

`charging_stations` **(för EV-optimering)**

```
CREATE TABLE charging_stations (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    name VARCHAR(255),
    address TEXT,
    latitude DECIMAL(10,8) NOT NULL,
    longitude DECIMAL(11,8) NOT NULL,
    charging_speed_kw DECIMAL(10,2),
    connector_types VARCHAR(50)[], -- ['CCS', 'CHAdeMO', 'Type2']
    is_available BOOLEAN DEFAULT true,
    price_per_kwh DECIMAL(10,4),
    created_at TIMESTAMP DEFAULT NOW()
);
```

---

7. API-endpoints

7.1 Autentisering

Metod	Endpoint	Beskrivning
POST	`/api/auth/login`	Logga in användare
POST	`/api/auth/logout`	Logga ut användare
GET	`/api/auth/me`	Hämta inloggad användare

7.2 Fordon

Metod	Endpoint	Beskrivning
GET	`/api/vehicles`	Lista alla fordon
GET	`/api/vehicles/:id`	Hämta specifikt fordon
POST	`/api/vehicles`	Skapa nytt fordon
PUT	`/api/vehicles/:id`	Uppdatera fordon
DELETE	`/api/vehicles/:id`	Ta bort fordon

7.3 Kunder

Metod	Endpoint	Beskrivning
GET	`/api/customers`	Lista alla kunder
GET	`/api/customers/:id`	Hämta specifik kund
POST	`/api/customers`	Skapa ny kund
PUT	`/api/customers/:id`	Uppdatera kund
DELETE	`/api/customers/:id`	Ta bort kund

7.4 Rutter

Metod	Endpoint	Beskrivning
GET	`/api/routes`	Lista alla rutter (med filter)
GET	`/api/routes/:id`	Hämta specifik rutt med stopp
POST	`/api/routes`	Skapa ny rutt
PUT	`/api/routes/:id`	Uppdatera rutt
DELETE	`/api/routes/:id`	Ta bort rutt
POST	`/api/routes/:id/optimize`	Optimera rutt
POST	`/api/routes/:id/duplicate`	Duplicera rutt

7.5 Stopp

Metod	Endpoint	Beskrivning
GET	`/api/routes/:routeId/stops`	Lista stopp för rutt
POST	`/api/routes/:routeId/stops`	Lägg till stopp
PUT	`/api/stops/:id`	Uppdatera stopp
DELETE	`/api/stops/:id`	Ta bort stopp
PUT	`/api/routes/:routeId/stops/reorder`	Ändra ordning på stopp

7.6 Optimering

Metod	Endpoint	Beskrivning
POST	`/api/optimize`	Optimera rutter för flera fordon
POST	`/api/optimize/preview`	Förhandsgranska optimering
GET	`/api/optimize/status/:jobId`	Hämta optimeringsstatus

7.7 Geocoding & Routing

Metod	Endpoint	Beskrivning
GET	`/api/geocode`	Geocoda adress till koordinater
GET	`/api/geocode/reverse`	Omvänd geocoding
POST	`/api/routing/directions`	Hämta vägbeskrivning mellan punkter
POST	`/api/routing/matrix`	Beräkna distansmatris

7.8 Statistik & Rapporter

Metod	Endpoint	Beskrivning
GET	`/api/stats/dashboard`	Hämta dashboard-statistik
GET	`/api/stats/co2`	Hämta CO2-statistik
GET	`/api/reports/daily`	Daglig rapport
GET	`/api/reports/export`	Exportera rapport (CSV/PDF)

7.9 AI-funktioner

Metod	Endpoint	Beskrivning
POST	`/api/ai/chat`	AI-chattassistent
POST	`/api/ai/predict-eta`	Prediktiv ETA-beräkning
POST	`/api/ai/scenario`	Scenarioanalys
POST	`/api/ai/suggestions`	Hämta AI-förslag

7.10 EV-specifika

Metod	Endpoint	Beskrivning
GET	`/api/charging-stations`	Lista laddstationer
POST	`/api/ev/range-check`	Kontrollera räckvidd för rutt
POST	`/api/ev/plan-charging`	Planera laddpauser

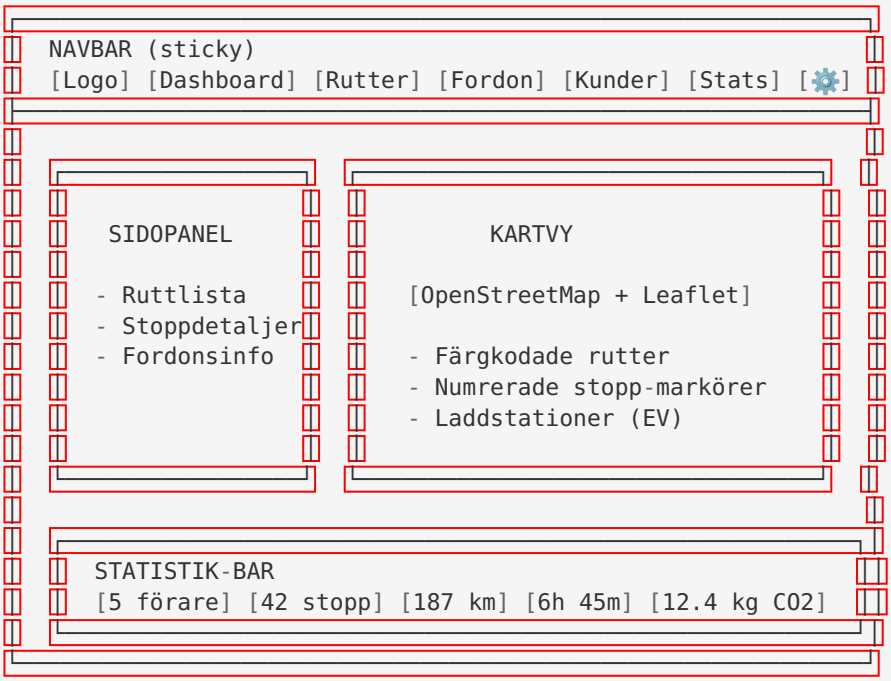
---

8. UI/UX-krav

8.1 Sidor/Vyer

Sida	URL	Beskrivning
Dashboard	`/`	Översikt med nyckeltal, dagens rutter, snabbåtgärder
Ruttplanering	`/routes`	Huvudvy med karta och ruttlista
Ny rutt	`/routes/new`	Skapa ny rutt med stopp
Ruttdetalj	`/routes/:id`	Visa/redigera specifik rutt
Fordon	`/vehicles`	Lista och hantera fordon
Kunder	`/customers`	Kundregister
Statistik	`/stats`	Detaljerad statistik och rapporter
CO2-dashboard	`/sustainability`	Hållbarhetsrapportering
Inställningar	`/settings`	Organisationsinställningar

8.2 Huvudlayout



8.3 Användarflöden

Flöde 1: Skapa och optimera rutt

```
flowchart TD
    A[Klicka "Ny rutt"] --> B[Välj datum och fordon]
    B --> C[Lägg till stopp]
    C --> D{Fler stopp?}
    D -->|Ja| C
    D -->|Nej| E[Klicka "Optimera"]
    E --> F[Visa optimerad rutt på karta]
    F --> G{Nöjd?}
    G -->|Ja| H[Spara rutt]
    G -->|Nej| I[Dra-och-släpp justering]
    I --> G
```

Flöde 2: Importera stopp från CSV

```
flowchart TD
    A[Klicka "Importera"] --> B[Välj CSV-fil]
    B --> C[Förhandsgranska data]
    C --> D[Mappa kolumner]
    D --> E[Validera adresser]
    E --> F{Alla giltiga?}
    F -->|Ja| G[Importera stopp]
    F -->|Nej| H[Visa fel, korrigera]
    H --> E
    G --> I[Tilldela till rutter]
```

Flöde 3: EV-ruttplanering

```
flowchart TD
    A[Välj elfordon] --> B[Ange batterinivå]
    B --> C[Lägg till stopp]
    C --> D[Optimera rutt]
    D --> E{Räcker batteriet?}
    E -->|Ja| F[Visa rutt]
    E -->|Nej| G[Föreslå laddstationer]
    G --> H[Välj laddstation]
    H --> I[Uppdatera rutt med laddpaus]
    I --> F
```

8.4 Responsivitet

Breakpoint	Bredd	Layout
Mobile	< 640px	Staplad layout, karta i full-skärm, slide-up panel för detaljer
Tablet	640-1024px	Sidopanel kollapsar till ikon-meny, karta tar mer plats
Desktop	> 1024px	Full layout med sidopanel och karta sida vid sida

8.5 Komponenter

Kärnkomponenter

- `<RouteMap />` – Leaflet-karta med rutter och stopp
- `<StopList />` – Drag-and-drop lista med stopp
- `<VehicleCard />` – Kort med fordonsinformation
- `<StatsBar />` – Statistik-rad med nyckeltal
- `<OptimizeButton />` – Knapp med laddningsindikator
- `<TimeWindowPicker />` – Tidsfönsterväljare
- `<AddressAutocomplete />` – Adressinmatning med autocomplete

Dashboard-komponenter

- `<KPICard />` – Nyckeltal med trend-indikator
- `<RouteChart />` – Linjediagram för ruttstatistik
- `<CO2Gauge />` – Cirkulär CO2-indikator
- `<TodayRoutes />` – Lista med dagens rutter
- `<QuickActions />` – Snabbåtgärdsknappar

Formulärkomponenter

- `<VehicleForm />` – Skapa/redigera fordon
- `<StopForm />` – Skapa/redigera stopp
- `<ImportWizard />` – Steg-för-steg import
- `<FilterPanel />` – Filtrera rutter/stopp

9. Filstruktur för Replit

```
routeai/
├── app/
│   ├── (auth)/
│   │   ├── login/
│   │   │   └── page.tsx
│   │   └── layout.tsx
│   ├── (dashboard)/
│   │   ├── layout.tsx
│   │   ├── page.tsx                    # Dashboard
│   │   ├── routes/
│   │   │   ├── page.tsx                # Ruttlista
│   │   │   ├── new/
│   │   │   │   └── page.tsx            # Ny rutt
│   │   │   └── [id]/
│   │   │       └── page.tsx            # Ruttdetalj
│   │   ├── vehicles/
│   │   │   └── page.tsx
│   │   ├── customers/
│   │   │   └── page.tsx
│   │   ├── stats/
│   │   │   └── page.tsx
│   │   └── settings/
│   │       └── page.tsx
│   ├── api/
│   │   ├── auth/
│   │   │   └── [...nextauth]/
│   │   │       └── route.ts
│   │   ├── vehicles/
│   │   │   ├── route.ts
│   │   │   └── [id]/
│   │   │       └── route.ts
│   │   ├── routes/
│   │   │   ├── route.ts
│   │   │   └── [id]/
│   │   │       ├── route.ts
│   │   │       ├── optimize/
│   │   │       │   └── route.ts
│   │   │       └── stops/
│   │   │           └── route.ts
│   │   ├── stops/
│   │   │   └── [id]/
│   │   │       └── route.ts
│   │   ├── customers/
│   │   │   └── route.ts
│   │   ├── optimize/
│   │   │   └── route.ts
│   │   ├── geocode/
│   │   │   └── route.ts
│   │   ├── routing/
│   │   │   └── route.ts
│   │   ├── stats/
│   │   │   └── route.ts
│   │   └── ai/
│   │       ├── chat/
│   │       │   └── route.ts
│   │       └── predict/
│   │           └── route.ts
│   ├── globals.css
│   └── layout.tsx
├── components/
│   ├── ui/                             # shadcn/ui komponenter
│   │   ├── button.tsx
```

```
│   │   ├── card.tsx
│   │   ├── input.tsx
│   │   └── ...
│   ├── map/
│   │   ├── RouteMap.tsx
│   │   ├── StopMarker.tsx
│   │   └── RoutePolyline.tsx
│   ├── routes/
│   │   ├── StopList.tsx
│   │   ├── RouteCard.tsx
│   │   └── OptimizeButton.tsx
│   ├── vehicles/
│   │   ├── VehicleCard.tsx
│   │   └── VehicleForm.tsx
│   ├── dashboard/
│   │   ├── KPICard.tsx
│   │   ├── StatsBar.tsx
│   │   └── CO2Gauge.tsx
│   └── layout/
│       ├── Navbar.tsx
│       ├── Sidebar.tsx
│       └── MobileNav.tsx
├── lib/
│   ├── db.ts                    # Prisma client
│   ├── utils.ts                 # Hjälpfunktioner
│   ├── routing.ts               # OSRM-integration
│   ├── geocoding.ts             # Nominatim-integration
│   ├── optimize.ts              # Optimeringsalgoritm
│   └── ai.ts                    # Abacus.AI-integration
├── hooks/
│   ├── useRoutes.ts
│   ├── useVehicles.ts
│   └── useOptimize.ts
├── types/
│   └── index.ts                 # TypeScript-typer
├── prisma/
│   ├── schema.prisma
│   └── seed.ts
├── public/
│   └── ...
├── .env.example
├── next.config.js
├── tailwind.config.ts
├── tsconfig.json
└── package.json
```

---

10. Kom igång i Replit

10.1 Steg 1: Skapa projekt

1. Skapa nytt Replit-projekt med **Next.js**-template
2. Välj **TypeScript**

10.2 Steg 2: Installera dependencies

```
npm install @prisma/client prisma
npm install react-leaflet leaflet @types/leaflet
npm install @tanstack/react-query
npm install zustand
npm install react-hook-form @hookform/resolvers zod
npm install framer-motion
npm install recharts
npm install date-fns
npm install lucide-react
npm install clsx tailwind-merge
npx shadcn-ui@latest init
```

10.3 Steg 3: Konfigurera databas

1. Skapa PostgreSQL-databas (Neon/Supabase/Railway)
2. Lägg till `DATABASE_URL` i Replit Secrets
3. Kör `npx prisma db push`

10.4 Steg 4: Miljövariabler

```
DATABASE_URL="postgresql://..."
NEXTAUTH_SECRET="..."
NEXTAUTH_URL="https://your-replit-url.repl.co"
ABACUS_API_KEY="..."
OSRM_URL="https://router.project-osrm.org"
```

---

11. Prioriterad Implementationsordning

Sprint 1 (Vecka 1-2): Grundläggande infrastruktur

- [ ] Projektsetup med Next.js + TypeScript + Tailwind
- [ ] Databas-schema och Prisma-setup
- [ ] Grundläggande layout (Navbar, Sidebar)
- [ ] Fordon CRUD
- [ ] Kunder CRUD

Sprint 2 (Vecka 3-4): Kartvy och rutter

- [ ] Leaflet-integration
- [ ] Rutter CRUD
- [ ] Stopp CRUD med geocoding
- [ ] Visa rutter på karta

Sprint 3 (Vecka 5-6): Optimering

- [ ] OSRM-integration
- [ ] Grundläggande ruttoptimering
- [ ] Drag-and-drop för stopp
- [ ] Statistik-bar

Sprint 4 (Vecka 7-8): Dashboard och statistik

- [ ] Dashboard med KPI:er
- [ ] Grundläggande rapporter
- [ ] CO2-beräkningar
- [ ] Import/Export

Sprint 5 (Vecka 9-10): AI och EV

- [ ] Abacus.AI-integration
- [ ] AI-chattassistent
- [ ] EV-räckviddsberäkning
- [ ] Laddstationsplanering

Dokumentet skapades baserat på analys av Routific.com och moderna UX/UI-principer.